Lab 03 - Azure Fundamentals for DevOps

Estimated timing: 70 minutes

Prerequisites

- The lab values provided by the instructor from the course setup script: the student name, the resource group **rg-<student>-usa**, the subscription ID and tenant ID, and the service principal client ID and client secret.
- A Bash terminal: **Git Bash** on Windows (installed in Lab 02A), **Terminal** on macOS, or a Linux shell.
- The Azure CLI. Exercise 1 installs it if needed.
- The lab files folder **labfiles/lab03**, which contains every command of this lab in **lab03-commands.sh**.

Objectives

This lab covers the following objectives:

- Install the Azure CLI and sign in with the user account and with the service principal
- Explore the assigned resource group and its role assignment
- Filter and format Azure CLI output with JMESPath queries
- Create a virtual network with two subnets and a network security group
- Create a storage account, a private container, and a blob
- Create a Linux virtual machine with an SSH key and connect to it
- Delete every resource that the lab created

Every resource in this lab is created with the Azure CLI. Lab 04 and Lab 05 create the same kind of resources with Terraform, which shows the difference between imperative commands and declarative code.

Exercise 1: Sign in and explore the lab environment

The exercise consists of the following tasks:

- Task 1: Install or verify the Azure CLI
- Task 2: Sign in and select the subscription
- Task 3: Explore the resource group and the role assignment
- Task 4: Query the output with JMESPath

Task 1: Install or verify the Azure CLI

1. Open the Bash terminal and run the following command:

   ```
   az version
   ```

2. If the command returns version information, continue with **Task 2**. Otherwise, install the Azure CLI with the command for the operating system, open a new terminal, and run `az version` again:

   ```
   # Windows (PowerShell)
   winget install --exact --id Microsoft.AzureCLI

   # macOS
   brew install azure-cli

   # Ubuntu or Debian
   curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
   ```

   Note: On Windows, the Azure CLI installed with winget also works in Git Bash after a new Git Bash window is opened.

Task 2: Sign in and select the subscription

1. Run the following command and complete the sign-in with the assigned account information:

   ```
   az login --service-principal \
   -u <app-id> \
   -p <client-secret> \
   --tenant <tenant-id>
   ```

2. If the account has access to several subscriptions, the command lists them. Select the course subscription by number, or run the following command with the subscription ID from the lab values:

```
az account set --subscription "<subscription-id>"
```

3. Verify the active subscription:

```
az account show --output table
```

4. Define the variables used in the rest of the lab. Replace `student01` with the assigned student name:

```
STUDENT="student01"
RG="rg-${STUDENT}-usa"
LOCATION="southcentralus"
TAGS="course=devops lab=03 owner=${STUDENT}"
```

Important: Variables exist only in the current terminal. If the terminal is closed, define them again before continuing.

Task 3: Explore the resource group and the role assignment

1. Show the resource group:

```
az group show --name "$RG" --output table
```

2. List the role assignments of the resource group:

```
az role assignment list \
--resource-group "$RG" \
--query "[].{principal:principalName, role:roleDefinitionName, scope:scope}" \
--output table
```

3. Note the assignment of the **Contributor** role to the service principal **sp-devops-<student>** at the scope of the resource group. The service principal can create and delete resources in this group, but it cannot assign roles or create resource groups.

4. List the resources in the group. The list is empty or contains only resources created by the course setup:

```
az resource list --resource-group "$RG" --output table
```

Task 4: Query the output with JMESPath

1. Compare the default JSON output with the table output:

```
az group show --name "$RG"
az group show --name "$RG" --output table
```

2. Select only some fields with a query:

```
az group show --name "$RG" --query "{name:name, location:location,
state:properties.provisioningState}"
```

3. List the names of the physical regions in the United States:

```
az account list-locations --query "[?metadata.geographyGroup=='US' &&
metadata.regionType=='Physical'].name" --output tsv
```

4. List the VM sizes in the region with at most two vCPUs and 4 GB of memory:

```
az vm list-sizes --location "$LOCATION" \
  --query "[?numberOfCores<=\`2\` && memoryInMB<=\`4096\`].{name:name,
cores:numberOfCores, memory:memoryInMB}" \
  --output table
```

5. Verify that **Standard_B1s** (1 vCPU, 1 GB) is in the list. Lab 03 uses this size for the virtual machine.

   Note: In JMESPath, literal numbers are written between backticks. In Bash, the backticks inside double quotes must be escaped with a backslash, as in the command above.

Exercise 2: Create the network

The exercise consists of the following tasks:

- Task 1: Create the virtual network and the subnets
- Task 2: Create a network security group that allows SSH only from the admin IP address

Task 1: Create the virtual network and the subnets

1. Create the virtual network with the first subnet:

```
az network vnet create --resource-group "$RG" --name vnet-lab03 --location
"$LOCATION" \
  --address-prefixes 10.20.0.0/16 --subnet-name snet-web --subnet-prefixes
10.20.1.0/24 \
  --tags $TAGS
```

2. Add the second subnet:

```
az network vnet subnet create --resource-group "$RG" --vnet-name vnet-lab03
\
  --name snet-data --address-prefixes 10.20.2.0/24
```

3. List the subnets and their address ranges:

```
az network vnet subnet list --resource-group "$RG" --vnet-name vnet-lab03 \
  --query "[].{name:name, prefix:addressPrefix}" --output table
```

Note: Each /24 subnet has 256 addresses, of which 251 are usable: Azure reserves the first four and the last address of every subnet.

Task 2: Create a network security group that allows SSH only from the admin IP address

1. Find the public IP address of the computer:

```
MY_IP=$(curl -s https://api.ipify.org)
echo "$MY_IP"
```

2. Create the network security group:

```
az network nsg create --resource-group "$RG" --name nsg-web-lab03 --
location "$LOCATION" --tags $TAGS
```

3. Add an inbound rule that allows SSH (TCP 22) only from this IP address:

```
az network nsg rule create --resource-group "$RG" --nsg-name nsg-web-lab03
\
  --name allow-ssh-from-my-ip --priority 100 --direction Inbound --access
Allow \
  --protocol Tcp --source-address-prefixes "${MY_IP}/32" --destination-
port-ranges 22
```

4. Associate the network security group with the web subnet:

```
az network vnet subnet update --resource-group "$RG" --vnet-name vnet-lab03
\
  --name snet-web --network-security-group nsg-web-lab03
```

5. List all rules of the network security group, including the default rules:

```
az network nsg rule list --resource-group "$RG" --nsg-name nsg-web-lab03 --
include-default \
  --query "[].{priority:priority, name:name, access:access,
source:sourceAddressPrefix}" --output table
```

6. Note that the default rule **DenyAllInBound** (priority 65500) blocks every other inbound connection from the internet.

   Important: Never allow SSH from **Any** or **Internet**. Open ports are scanned within minutes of their creation.

Exercise 3: Create a storage account

The exercise consists of the following tasks:

- Task 1: Create the storage account
- Task 2: Create a container and upload a blob

Task 1: Create the storage account

1. Define a globally unique storage account name (3 to 24 lowercase letters and numbers):

```
ST="st${STUDENT}lab03${RANDOM}"
echo "$ST"
```

2. Create the storage account with secure settings:

```
az storage account create --resource-group "$RG" --name "$ST" --location
"$LOCATION" \
  --sku Standard_LRS --kind StorageV2 --min-tls-version TLS1_2 \
  --allow-blob-public-access false --tags $TAGS
```

3. Verify the settings:

```
az storage account show --resource-group "$RG" --name "$ST" \
  --query "{name:name, sku:sku.name, tls:minimumTlsVersion,
publicBlob:allowBlobPublicAccess}" --output table
```

Task 2: Create a container and upload a blob

1. Create a private container. The option `--auth-mode key` uses the account key, which the Contributor role can read:

```
az storage container create --account-name "$ST" --name notes --auth-mode
key
```

2. Create a small file and upload it:

```
echo "Hello from Lab 03" > hello.txt
az storage blob upload --account-name "$ST" --container-name notes \
  --name hello.txt --file hello.txt --auth-mode key
```

3. List the blobs in the container:

```
az storage blob list --account-name "$ST" --container-name notes --auth-mode key \
  --query "[].{name:name, size:properties.contentLength}" --output table
```

Note: With `--auth-mode login`, the Azure CLI uses Microsoft Entra ID for data access, which requires a data role such as **Storage Blob Data Contributor**. The lab identities have no data roles, so the lab uses the account key.

Exercise 4: Create a Linux virtual machine with an SSH key

The exercise consists of the following tasks:

- Task 1: Generate an SSH key pair
- Task 2: Create the virtual machine
- Task 3: Connect with SSH and review the created resources

Task 1: Generate an SSH key pair

1. Verify that the OpenSSH client is available:

   ```
   ssh -V
   ```

2. Generate an Ed25519 key pair for the lab. Press **Enter** twice to leave the passphrase empty:

   ```
   ssh-keygen -t ed25519 -f ~/.ssh/lab03_ed25519 -C "${STUDENT}-lab03"
   ```

3. Display the public key. This is the only part of the key pair that is given to the virtual machine:

   ```
   cat ~/.ssh/lab03_ed25519.pub
   ```

   Important: The private key **~/.ssh/lab03_ed25519** stays on the computer. It is never copied, shared, or committed to a repository.

Task 2: Create the virtual machine

1. Create a Standard_B1s Ubuntu 24.04 virtual machine in the web subnet. The option `--nsg ""` skips a network security group on the network interface, because the subnet already has one:

   ```
   az vm create --resource-group "$RG" --name vm-lab03 --location "$LOCATION"
   \
     --image Ubuntu2404 --size Standard_B1s \
     --vnet-name vnet-lab03 --subnet snet-web --nsg "" \
     --public-ip-sku Standard --admin-username azureuser \
     --ssh-key-values ~/.ssh/lab03_ed25519.pub --tags $TAGS
   ```

2. Wait until the command completes (about two minutes). Note the **publicIpAddress** value in the output.

   Note: If the size is not available in the region, the command reports an error such as **SkuNotAvailable**. Use the query from Exercise 1, Task 4 to select another small size, such as **Standard_B1ms**, and run the command again with that size.

Task 3: Connect with SSH and review the created resources

1. Store the public IP address in a variable and connect:

```
VM_IP=$(az vm show --resource-group "$RG" --name vm-lab03 --show-details --
query publicIps --output tsv)
ssh -i ~/.ssh/lab03_ed25519 "azureuser@${VM_IP}"
```

2. Type `yes` to accept the host key fingerprint at the first connection.
3. In the SSH session, run the following commands to display information about the virtual machine, and then close the session:

```
hostname
cat /etc/os-release | head -2
cat ~/.ssh/authorized_keys
exit
```

4. Back on the computer, list the resources that the `az vm create` command created:

```
az resource list --resource-group "$RG" --query "[?contains(name, 'vm-
lab03')].{name:name, type:type}" --output table
```

5. Note that one command created four resources: the virtual machine, the network interface, the public IP address, and the OS disk.

Exercise 5: Clean up the resources

The exercise consists of the following tasks:

- Task 1: Delete the lab resources

Task 1: Delete the lab resources

1. Delete the virtual machine:

```
az vm delete --resource-group "$RG" --name vm-lab03 --yes
```

2. Delete the network interface, the public IP address, and the OS disk that remain after the virtual machine is deleted:

```
for type in Microsoft.Network/networkInterfaces
Microsoft.Network/publicIPAddresses Microsoft.Compute/disks; do
  az resource list --resource-group "$RG" --resource-type "$type" \
    --query "[?contains(name, 'vm-lab03')].id" --output tsv |
    while read -r id; do az resource delete --ids "$id"; done
done
```

3. Delete the virtual network, the network security group, and the storage account:

```
az network vnet delete --resource-group "$RG" --name vnet-lab03
az network nsg delete --resource-group "$RG" --name nsg-web-lab03
az storage account delete --resource-group "$RG" --name "$ST" --yes
```

4. Verify that no resource with the tag **lab=03** remains:

```
az resource list --resource-group "$RG" --query "[?tags.lab=='03'].name" --
output table
```

   Important: Do not delete the resource group itself. It belongs to the course environment and is used by every following lab.

Review

The following table summarizes the Azure CLI commands practiced in this lab. Lab 04 creates the same kind of resources with Terraform.

Task	Azure CLI command
Sign in (user, service principal)	`az login`, `az login --service-principal`
Select the subscription	`az account set --subscription <id>`
Filter and format output	`--query "<JMESPath>"`, `--output table` or `tsv`
Virtual network and subnets	`az network vnet create`, `az network vnet subnet create`
Network security group and rule	`az network nsg create`, `az network nsg rule create`
Storage account and container	`az storage account create`, `az storage container create --auth-mode key`
Linux virtual machine	`az vm create --image Ubuntu2404 --size Standard_B1s --ssh-key-values <file>`
Clean up	`az vm delete`, `az resource delete`, `az network vnet delete`